

Make a Ticketing System

Grade: B

Status: Pass (automatic by LLM)

Feedback

Day 1 Lakebase Homework Grade

Final Score: 87/100

Rubric Breakdown

CategoryScoreMaximumFeedbackLakebase schema2020Well-modeled schema provided in schema.sql and initialize_db.sql with UUID primary keys, relevant fields, and a foreign key from ticket_messages.ticket_id -> tickets.ticket_id (with ON DELETE CASCADE).Sample data1010initialize_db.sql inserts three tickets with open/in_progress/resolved statuses and at least two messages per ticket. Note: sample_data.sql and setup scripts contain inconsistencies (see "What to Improve").Reading from Lakebase2020App code (app.py) clearly queries Lakebase via psycopg2 to list tickets and fetch per-ticket messages; no hard-coded data observed.Creating data2020Routes and functions to create tickets (/tickets) and add messages (/tickets/<id>/messages) perform INSERTs and commit, indicating persistence.Updating ticket status1010Implemented via /tickets/<id>/status route and update_ticket_status() UPDATE statement with NOW() for updated_at.Deployment510Evidence of Databricks Apps deployment intent via app.yaml and README instructions, but no App URL or deployment screenshot provided to verify a live deployment.Submission and reflection210No App URL, no repository URL, no screenshots. A short reflection is present in TICKETS_README.md.

What You Did Well

- Strong, relational Lakebase schema with proper UUID primary keys, foreign key constraints, and helpful indexes.
- Clear end-to-end CRUD flows implemented against Lakebase: list tickets, view ticket details with messages, create tickets, add messages, and update status.
- Secure secret handling using Databricks Secrets and base64 decoding, avoiding hard-coded credentials in app code.

What to Improve

- Sample data scripts are inconsistent:
- sample_data.sql inserts messages with gen_random_uuid() for ticket_id, which won't relate to the inserted tickets.
- setup_tickets.py and setup_tickets_databricks.py insert messages using ticket IDs 1/2/3, which won't work with UUID PKs. Align these scripts with initialize_db.sql's approach (selecting ticket_id by title) or fetch the UUIDs programmatically before inserting messages.
- Deployment evidence is missing. Please provide:
- Databricks App URL
- Screenshot of the deployed app
- Screenshot of Lakebase tables and sample rows

- Submission completeness: Include a source-code repository URL (GitHub/Workspace path link). This will satisfy the remaining submission checklist items.

Instructor Summary

You implemented a complete, Lakebase-backed ticketing app with solid schema design and full CRUD + status update workflows. The code clearly reads from and writes to Lakebase, satisfying persistence and interaction requirements. Missing deployment evidence and inconsistent sample data scripts are the main gaps. Provide the App URL, screenshots, and repo link, and fix the sample data scripts to fully meet all requirements.

Evidence Limitations

- No Databricks App URL provided.
- No repository URL provided.
- No screenshots of the deployed application or Lakebase tables.
- Deployment not verifiable from the submission.

Security Check

No exposed credentials found. Secrets are retrieved from Databricks Secret Scopes. The local test mock includes a base64 string of a placeholder URL ("role:pass@host"), not a real secret.